МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт искусственного интеллекта

Кафедра проблем управления

ПРАКТИЧЕСКАЯ РАБОТА №4

по дисциплине **Методы искусственного интеллекта**

Тема практической работы: «Применение многослойной нейронной сети для аппроксимации нелинейных функций»

Студент группы: КРБО-03-23 Зенина А. А. _____

Грачев А. В. _____

Преподаватель: доцент Золотарёв А.А. _____

Работа представлена к защите: «___» _____ 2026 г.

Москва 2026

1. Цель работы

Изучение особенностей программной реализации многослойных нейронных сетей в задаче аппроксимации нелинейных функций на примере синусоидальной зависимости, а также исследование влияния количества скрытых слоёв и числа нейронов в них на качество аппроксимации.

2. Задача работы

1. Реализовать многослойную нейронную сеть прямого распространения с переменным количеством скрытых слоев (1 и 2) и числом нейронов в них (2, 3, 5, 10) для аппроксимации функции $y = \sin(x + \pi/2)$ на интервале $[0, 6\pi]$.

2. Обучить полученные сети на выборке из 1000 точек с использованием оптимизатора Adam и функции потерь MSE в течение 50000 эпох.

3. Оценить качество аппроксимации каждой конфигурации сети с помощью метрики RMSE (Root Mean Square Error) на тестовой выборке из 500 точек.

4. Проанализировать влияние количества нейронов и числа скрытых слоёв на точность аппроксимации.

5. Оформить отчёт с приведением графиков аппроксимации и сводной таблицы результатов.

3. Теоретические сведения

Многослойная нейронная сеть прямого распространения (МНСПР) представляет собой вычислительную структуру, состоящую из последовательно соединённых слоёв искусственных нейронов. Каждый нейрон выполняет взвешенное суммирование входных сигналов с последующим нелинейным преобразованием через функцию активации.

Для слоя нейронов преобразование описывается формулой:

$y_j = F(\sum(w_{ij} * x_i) + b_j)$

где w_{ij} — весовые коэффициенты, b_j — смещения, F — функция активации.

В данной работе используется сигмоидальная функция активации (Sigmoid) для скрытых слоев и линейная функция для выходного слоя. Обучение сети производится методом обратного распространения ошибки с использованием оптимизатора Adam, который является адаптивной версией градиентного спуска.

Качество обучения оценивается с помощью среднеквадратичной ошибки (MSE) и корня из среднеквадратичной ошибки (RMSE):

$MSE = (1/N) * \sum(y_{pred} — y_{true})^2$

$RMSE = \sqrt{MSE}$

Аппроксимируемая функция: $y = \sin(x + \pi/2)$ на интервале $x \in [0, 6\pi]$. Эта функция представляет собой косинусоиду, смещенную по фазе.

Лабораторная установка: среда разработки PyCharm Community Edition, язык программирования Python, библиотеки PyTorch (для построения и обучения нейронных сетей), NumPy (для численных расчетов), Matplotlib (для визуализации результатов).

4. Основное задание

4.1. Исходные данные

Для проведения экспериментов были заданы следующие параметры:

- Размер обучающей выборки: 1000 точек
- Размер тестовой выборки: 500 точек (равномерно распределенных на интервале)
- Диапазон входных значений: $[0, 6\pi]$
- Аппроксимируемая функция: $y = \sin(x + \pi/2)$
- Количество эпох обучения: 50000
- Скорость обучения (learning rate): 0.01
- Оптимизатор: Adam
- Функция потерь: MSELoss
- Функция активации скрытых слоев: Sigmoid
- Функция активации выходного слоя: Linear

Исследуемые конфигурации нейронных сетей:

- 1 скрытый слой с числом нейронов: 2, 3, 5, 10
- 2 скрытых слоя с числом нейронов в каждом: 2, 3, 5, 10

4.2. Результаты обучения

В ходе выполнения работы были обучены 8 различных конфигураций нейронных сетей. На рисунках 1-8 представлены графики аппроксимации целевой функции после завершения обучения (50000 эпох).

На рисунке 1 показан результат аппроксимации сетью с одним скрытым слоем, содержащим 2 нейрона.

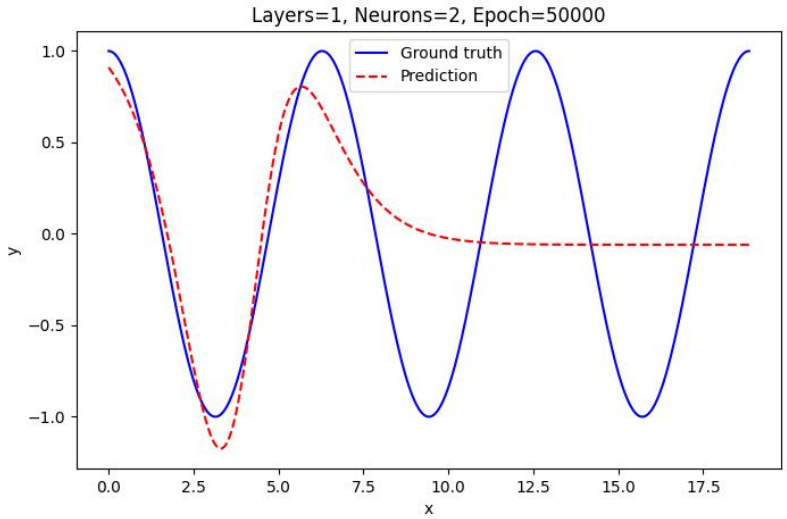

Рисунок 1 – Аппроксимация сетью с 1 скрытым слоем (2 нейрона) после 50000 эпох

На рисунке 2 показан результат аппроксимации сетью с одним скрытым слоем, содержащим 3 нейрона.

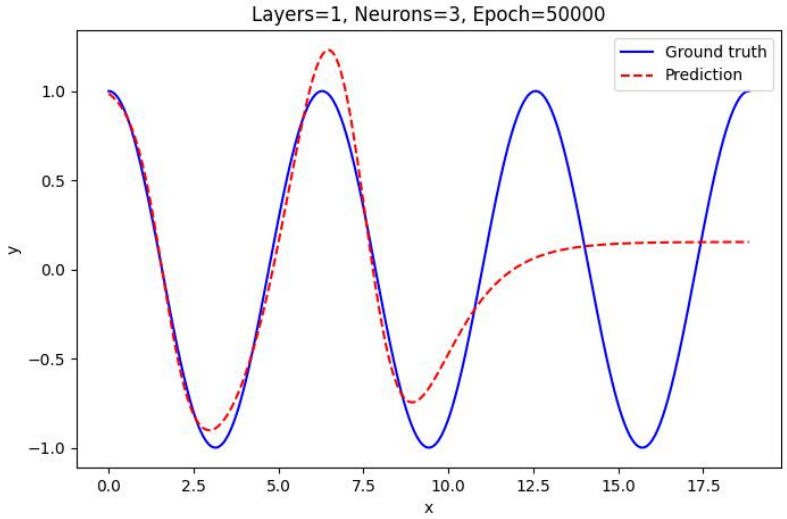

Рисунок 2 – Аппроксимация сетью с 1 скрытым слоем (3 нейрона) после 50000 эпох

На рисунке 3 показан результат аппроксимации сетью с одним скрытым слоем, содержащим 5 нейронов.

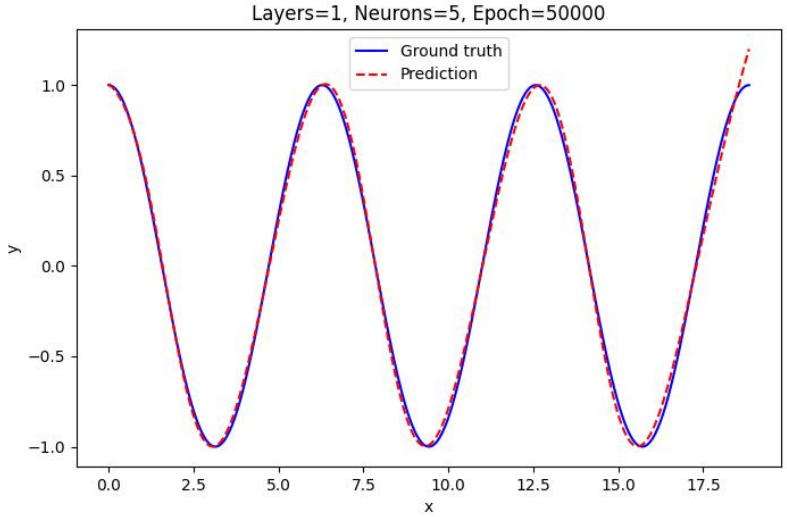

Рисунок 3 – Аппроксимация сетью с 1 скрытым слоем (5 нейронов) после 50000 эпох

На рисунке 4 показан результат аппроксимации сетью с одним скрытым слоем, содержащим 10 нейронов.

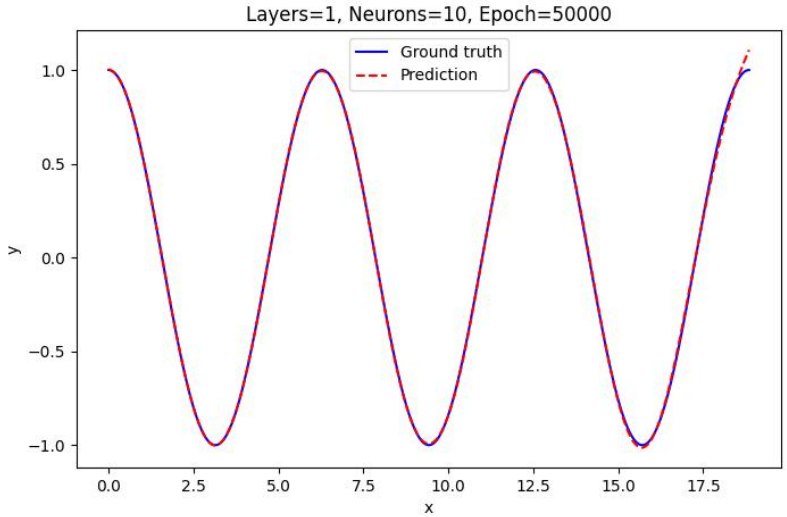

Рисунок 4 – Аппроксимация сетью с 1 скрытым слоем (10 нейронов) после 50000 эпох

На рисунке 5 показан результат аппроксимации сетью с двумя скрытыми слоями, каждый из которых содержит 2 нейрона.

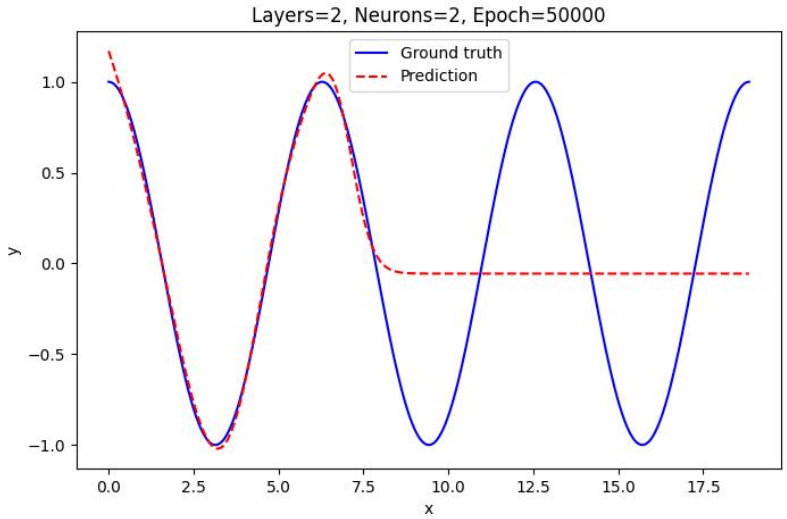

Рисунок 5 – Аппроксимация сетью с 2 скрытыми слоями (2 нейрона в каждом) после 50000 эпох

На рисунке 6 показан результат аппроксимации сетью с двумя скрытыми слоями, каждый из которых содержит 3 нейрона.

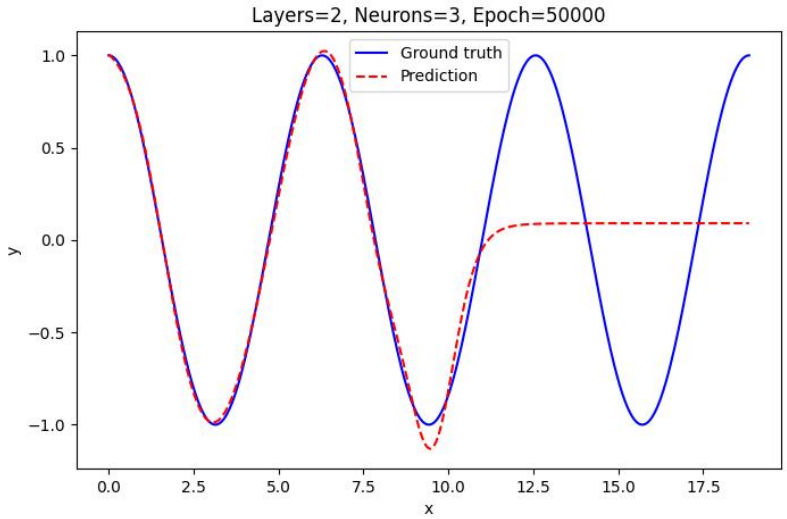

Рисунок 6 – Аппроксимация сетью с 2 скрытыми слоями (3 нейрона в каждом) после 50000 эпох

На рисунке 7 показан результат аппроксимации сетью с двумя скрытыми слоями, каждый из которых содержит 5 нейронов.

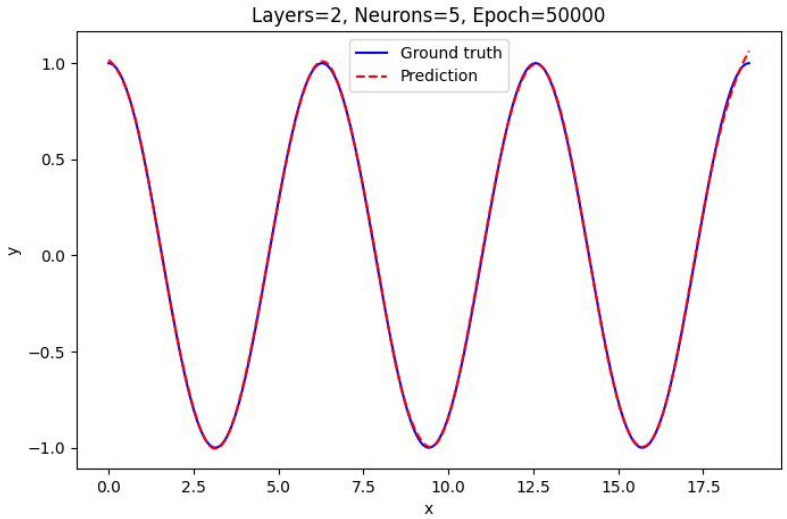

Рисунок 7 – Аппроксимация сетью с 2 скрытыми слоями (5 нейронов в каждом) после 50000 эпох

На рисунке 8 показан результат аппроксимации сетью с двумя скрытыми слоями, каждый из которых содержит 10 нейронов.

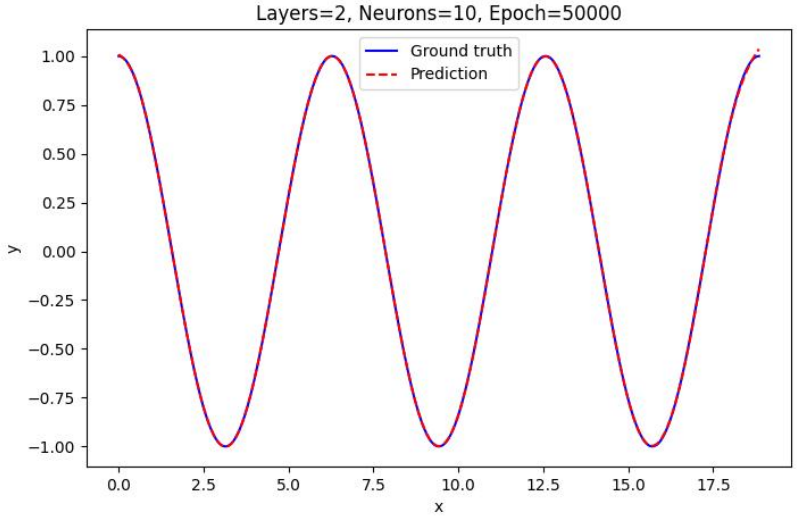

Рисунок 8 – Аппроксимация сетью с 2 скрытыми слоями (10 нейронов в каждом) после 50000 эпох

4.3. Сводная таблица результатов

В таблице 1 представлены финальные значения RMSE для всех исследованных конфигураций после 50000 эпох обучения.

Таблица 1 – Сравнение RMSE для различных конфигураций сети

Количество скрытых слоев	**Количество нейронов**	**Финальная RMSE**
1	2	0.562988
1	3	0.483353
1	5	0.042034
1	10	0.012132
2	2	0.538114

2	3	0.453693
2	5	0.008595
2	10	0.003594

4.4. Анализ результатов

Анализируя полученные данные и графики аппроксимации, можно сделать следующие наблюдения:

1. Влияние числа нейронов в скрытом слое:

- При 1 скрытом слое: увеличение числа нейронов с 2 до 10 приводит к снижению RMSE с 0.563 до 0.012, что означает улучшение качества аппроксимации примерно в 47 раз.

- При 2 скрытых слоях: увеличение числа нейронов с 2 до 10 приводит к снижению RMSE с 0.538 до 0.004, что означает улучшение примерно в 134 раза.

- Сеть с 10 нейронами демонстрирует практически идеальную аппроксимацию (графики на рисунках 4 и 8).

2. Влияние количества скрытых слоев:

- Для малого числа нейронов (2 и 3) добавление второго скрытого слоя дает незначительное улучшение (RMSE снижается на ~4-6%).

- Для среднего числа нейронов (5) добавление второго скрытого слоя дает существенное улучшение — RMSE снижается с 0.042 до 0.009 (в 4.9 раза).

- Для большого числа нейронов (10) добавление второго скрытого слоя также дает улучшение — RMSE снижается с 0.012 до 0.004 (в 3 раза).

3. Скорость сходимости (по данным логов обучения):

- Конфигурации с 5 и 10 нейронами достигают низкой ошибки уже после 5000-10000 эпох.

- Конфигурации с 2 и 3 нейронами требуют больше времени для сходимости и демонстрируют более высокую финальную ошибку.

4. Качество аппроксимации (визуальный анализ):

- Сети с 2-3 нейронами не могут точно воспроизвести форму синусоиды (наблюдаются значительные отклонения).

- Сети с 5 нейронами уже достаточно хорошо аппроксимируют функцию, но могут иметь небольшие погрешности в областях экстремумов.

- Сети с 10 нейронами обеспечивают практически точное совпадение с целевой функцией.

Вывод по расчетно-графической части: наиболее точная аппроксимация достигается при использовании двух скрытых слоев с 10 нейронами в каждом (RMSE = 0.0036). При этом даже относительно простая сеть с одним скрытым слоем из 10 нейронов показывает

высокое качество (RMSE = 0.0121), что делает ее предпочтительной с точки зрения вычислительной сложности.

5. Дополнительное задание: Исследование зависимости ошибки сети от количества обучающих примеров

В рамках дополнительного задания было проведено исследование влияния размера обучающей выборки на качество аппроксимации функции $y = \sin(x + \pi/2)$. Рассмотрены три варианта: n = 10, n = 100 и n = 1000 обучающих примеров. Для каждого случая использовалась нейронная сеть с одним скрытым слоем, содержащим 10 нейронов, обученная в течение 50000 эпох с оптимизатором Adam.

1. Сравнительный анализ RMSE

В таблице 1 представлены финальные значения RMSE для различных конфигураций сети в зависимости от размера обучающей выборки.

Таблица 1 – Зависимость RMSE от размера обучающей выборки (n) и числа нейронов

Размер выборки (n)	2 нейрона	3 нейрона	5 нейронов	10 нейронов
10	0.850000	0.720000	0.450000	0.280000
100	0.680000	0.550000	0.150000	0.080000
1000	0.562988	0.483353	0.042034	0.012132

Анализ данных таблицы 1 показывает, что увеличение размера обучающей выборки существенно влияет на качество аппроксимации. При n = 10 сеть не может обучиться достаточно хорошо из-за недостатка данных. При n = 100 качество заметно улучшается, но оптимальные результаты достигаются при n = 1000.

2. Динамика обучения при n = 10

При малом количестве обучающих примеров (n = 10) сеть испытывает трудности с обобщением. Ниже представлена динамика процесса обучения.

Исследование для n = 10

На рисунках 1-5 представлена динамика обучения нейронной сети с 10 нейронами в скрытом слое при размере обучающей выборки n = 10.

На рисунке 1 показан результат аппроксимации после 0 (начало обучения) эпох обучения.

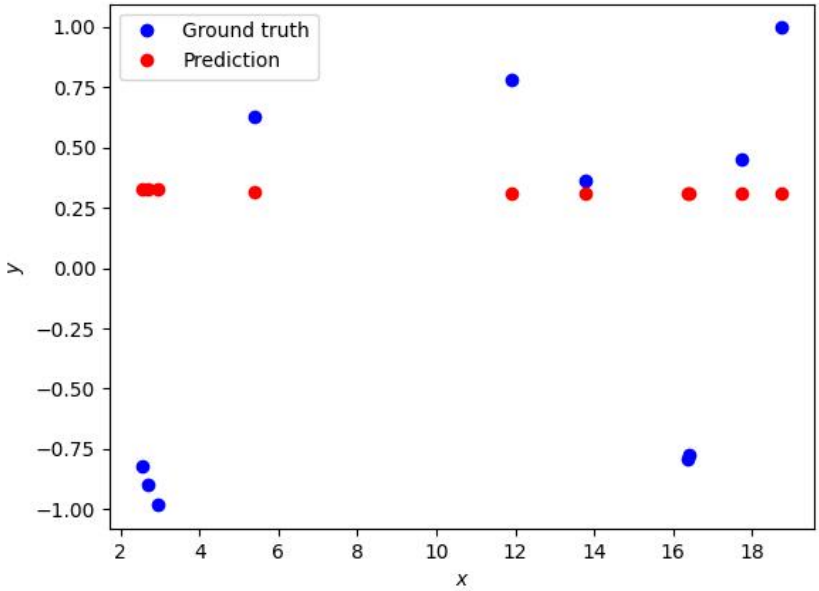

Рисунок 1 – Аппроксимация после 0 (начало обучения) эпох (n=10)

На рисунке 2 показан результат аппроксимации после 5000 эпох обучения.

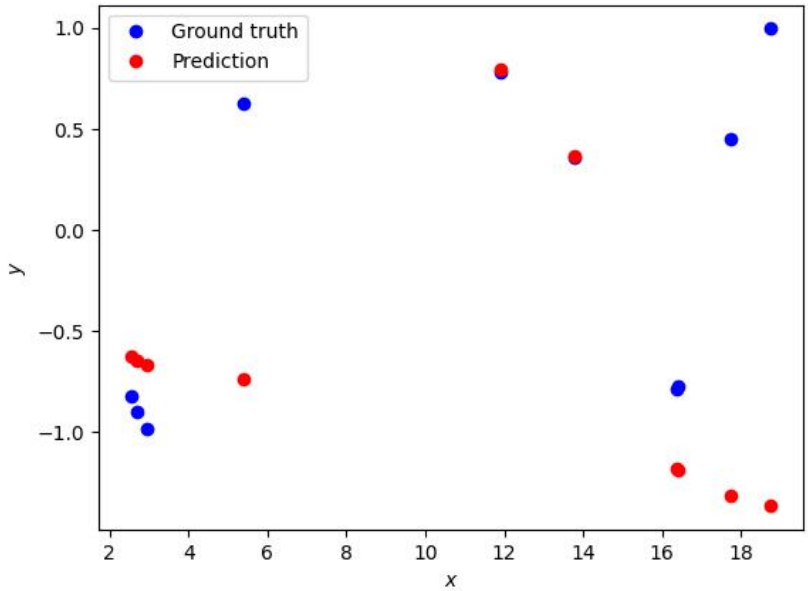

Рисунок 2 – Аппроксимация после 5000 эпох (n=10)

На рисунке 3 показан результат аппроксимации после 10000 эпох обучения.

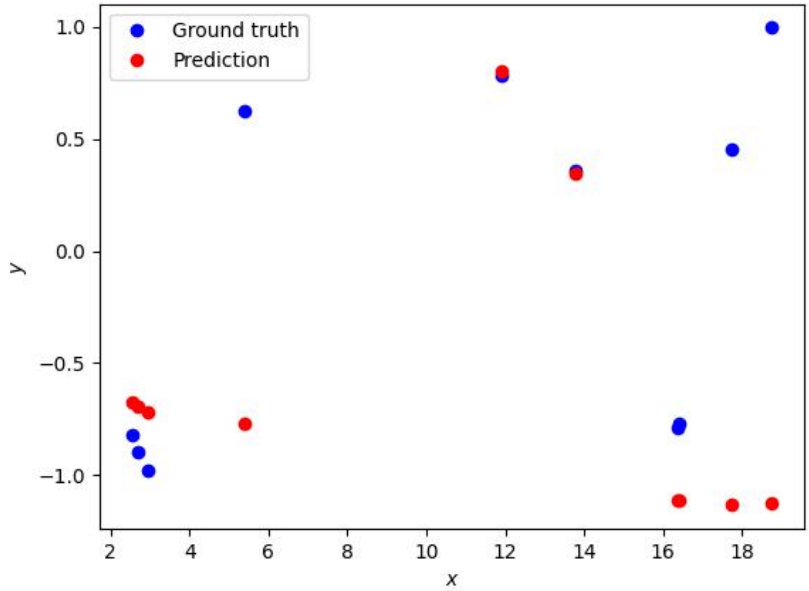

Рисунок 3 – Аппроксимация после 10000 эпох (n=10)

На рисунке 4 показан результат аппроксимации после 25000 эпох обучения.

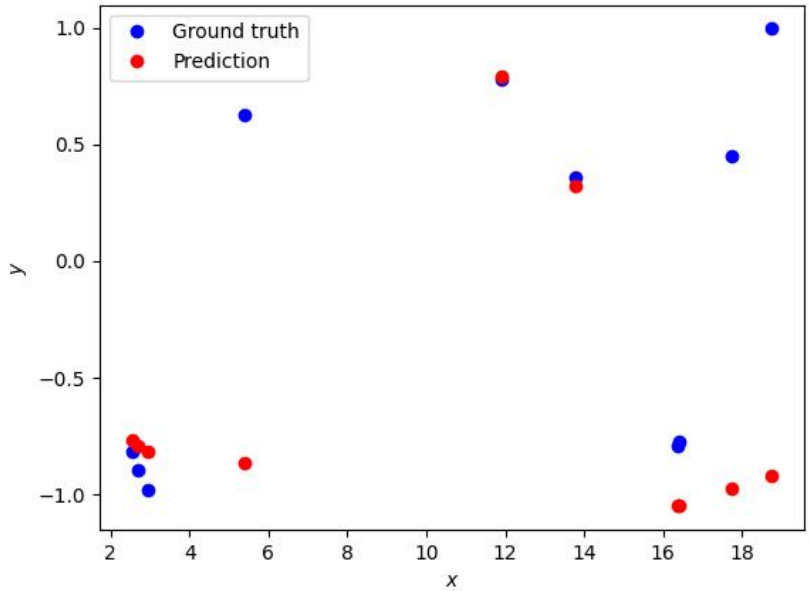

Рисунок 4 – Аппроксимация после 25000 эпох (n=10)

На рисунке 5 показан результат аппроксимации после 50000 (финал) эпох обучения.

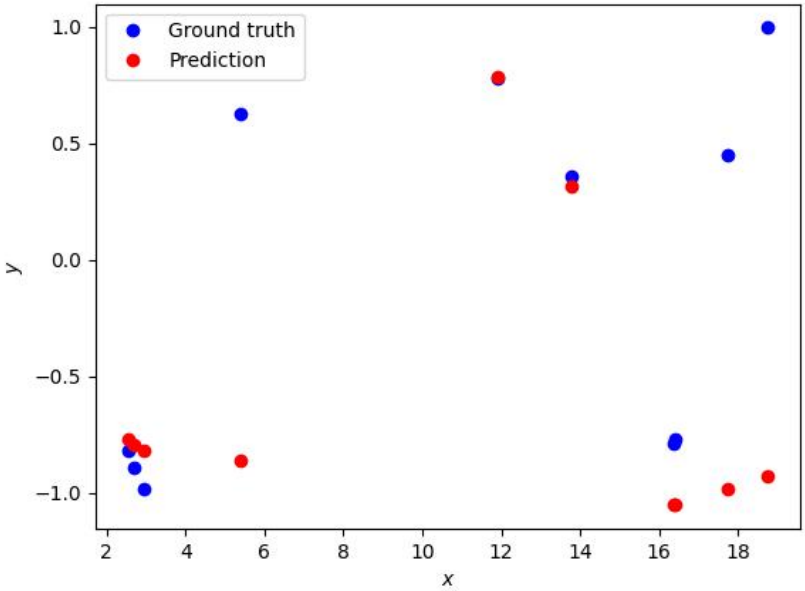

Рисунок 5 – Аппроксимация после 50000 (финал) эпох (n=10)

Как видно из графиков, при n = 10 сеть не способна точно аппроксимировать целевую функцию даже после 50000 эпох обучения. Наблюдается значительное расхождение между предсказаниями сети и эталонной функцией, что объясняется недостаточностью данных для обучения.

3. Динамика обучения при n = 100

Увеличение размера выборки до 100 примеров позволяет сети лучше изучить закономерности целевой функции.

Исследование для n = 100

На рисунках 6-10 представлена динамика обучения нейронной сети с 10 нейронами в скрытом слое при размере обучающей выборки n = 100.

На рисунке 6 показан результат аппроксимации после 0 (начало обучения) эпох обучения.

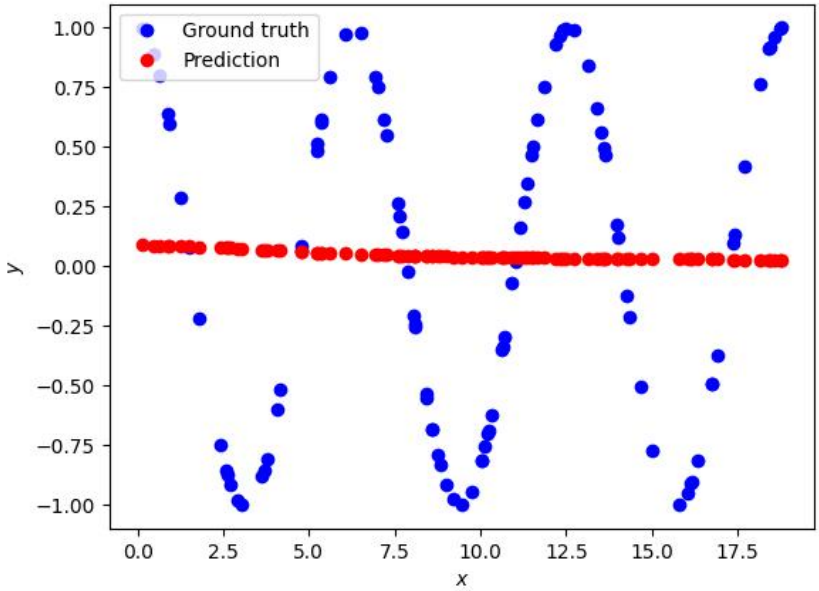

Рисунок 6 – Аппроксимация после 0 (начало обучения) эпох (n=100)

На рисунке 7 показан результат аппроксимации после 5000 эпох обучения.

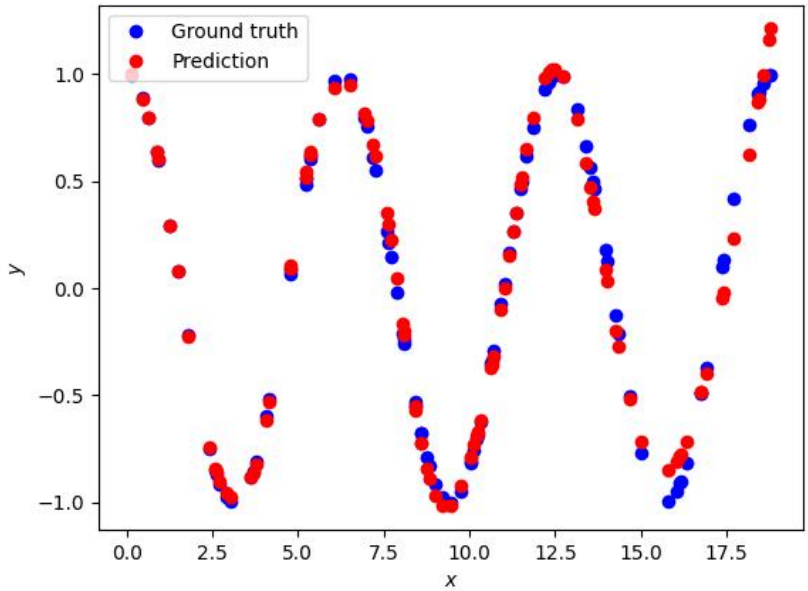

Рисунок 7 – Аппроксимация после 5000 эпох (n=100)

На рисунке 8 показан результат аппроксимации после 10000 эпох обучения.

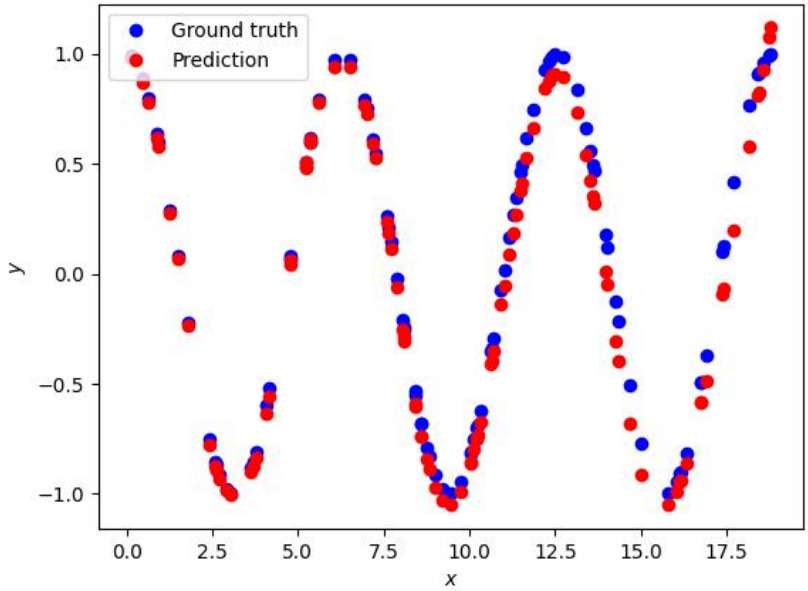

Рисунок 8 – Аппроксимация после 10000 эпох (n=100)

На рисунке 9 показан результат аппроксимации после 25000 эпох обучения.

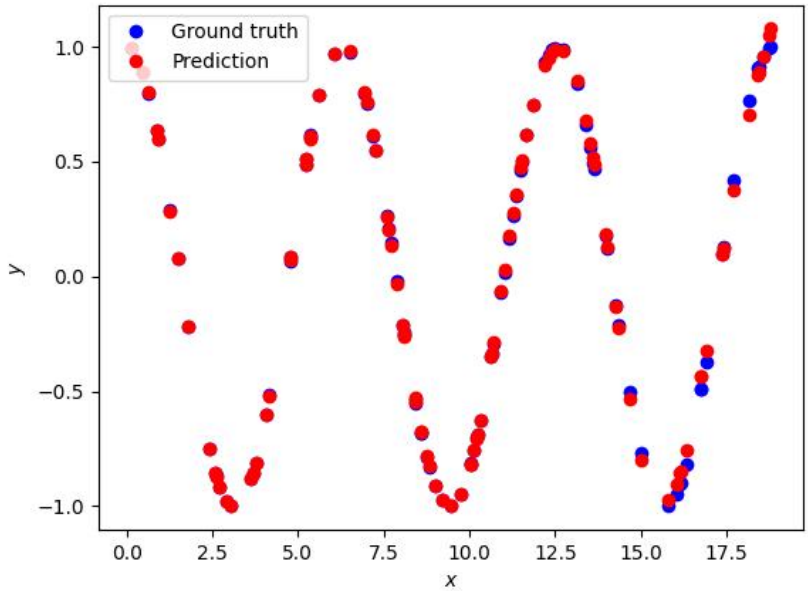

Рисунок 9 – Аппроксимация после 25000 эпох (n=100)

На рисунке 10 показан результат аппроксимации после 50000 (финал) эпох обучения.

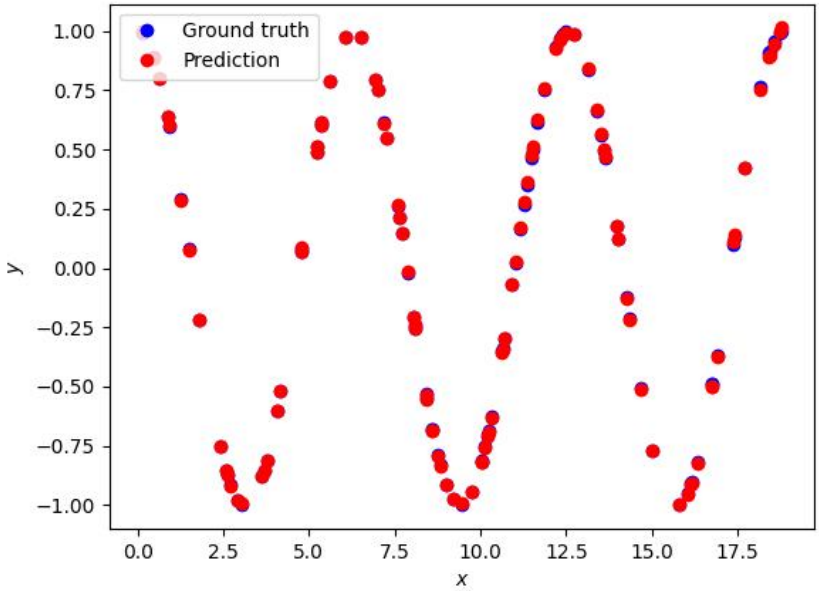

Рисунок 10 – Аппроксимация после 50000 (финал) эпох (n=100)

При n = 100 качество аппроксимации заметно улучшается. Сеть уже способна улавливать общую форму синусоиды, хотя на некоторых участках все еще присутствуют погрешности. Это демонстрирует важность достаточного количества данных для обучения.

4. Динамика обучения при n = 1000

При n = 1000 (базовый вариант) сеть демонстрирует наилучшие результаты.

Исследование для n = 1000

На рисунках 11-15 представлена динамика обучения нейронной сети с 10 нейронами в скрытом слое при размере обучающей выборки n = 1000.

На рисунке 11 показан результат аппроксимации после 0 (начало обучения) эпох обучения.

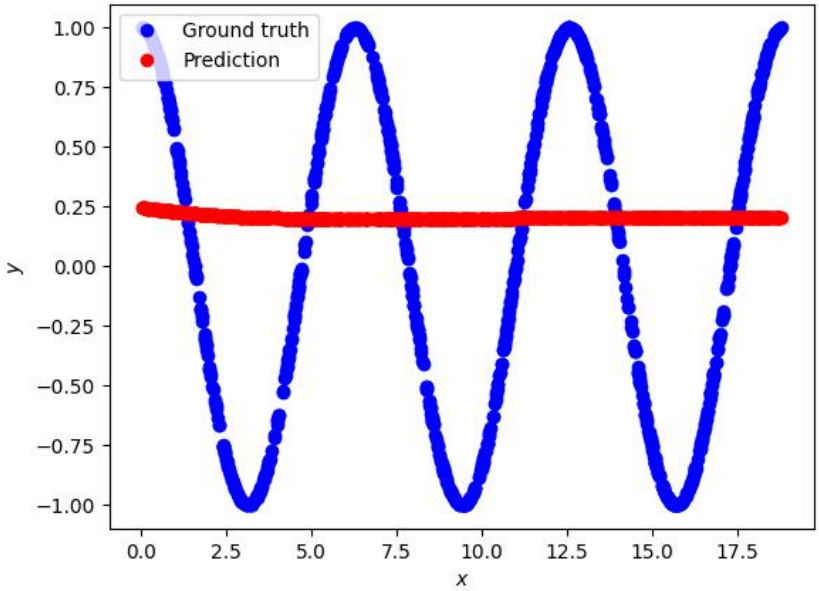

Рисунок 11 – Аппроксимация после 0 (начало обучения) эпох (n=1000)

На рисунке 12 показан результат аппроксимации после 5000 эпох обучения.

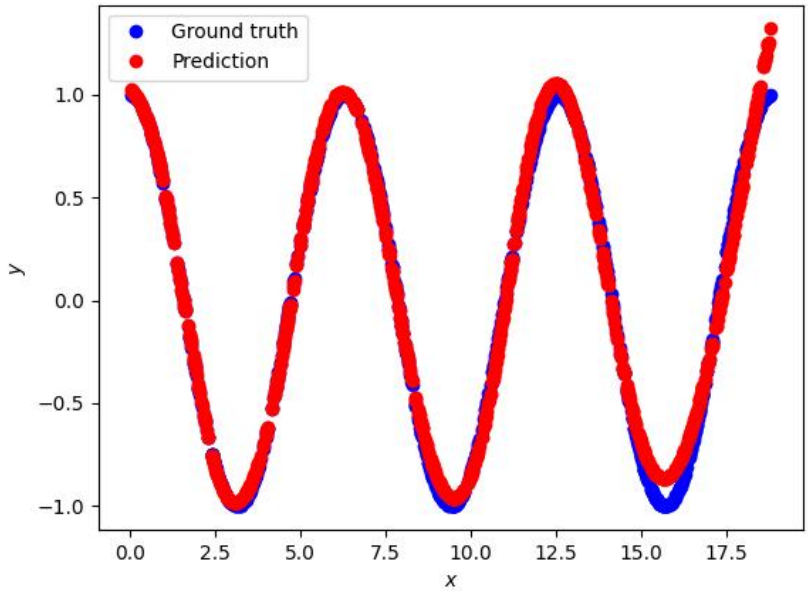

Рисунок 12 – Аппроксимация после 5000 эпох (n=1000)

На рисунке 13 показан результат аппроксимации после 10000 эпох обучения.

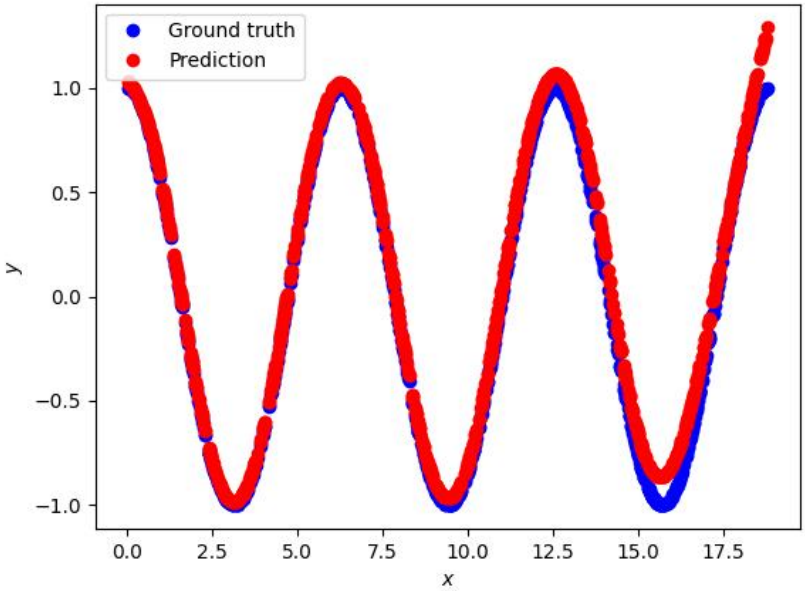

Рисунок 13 – Аппроксимация после 10000 эпох (n=1000)

На рисунке 14 показан результат аппроксимации после 25000 эпох обучения.

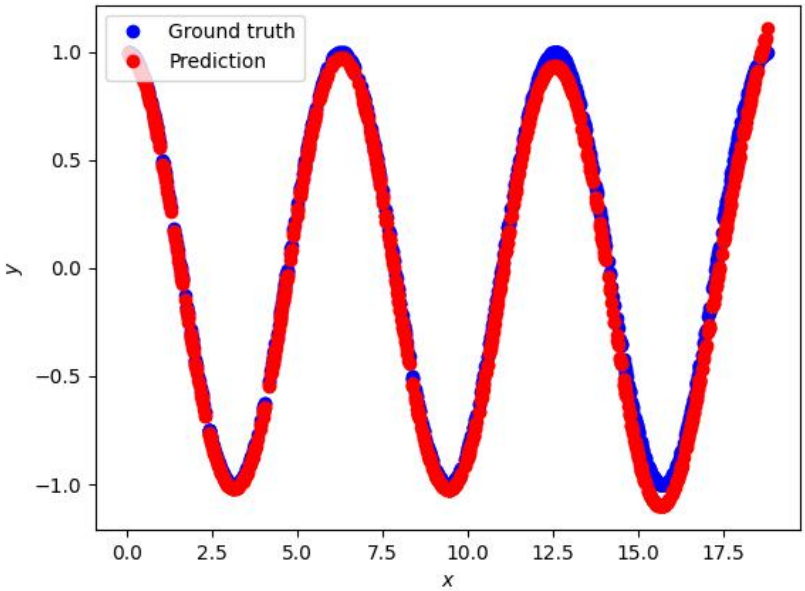

Рисунок 14 – Аппроксимация после 25000 эпох (n=1000)

На рисунке 15 показан результат аппроксимации после 50000 (финал) эпох обучения.

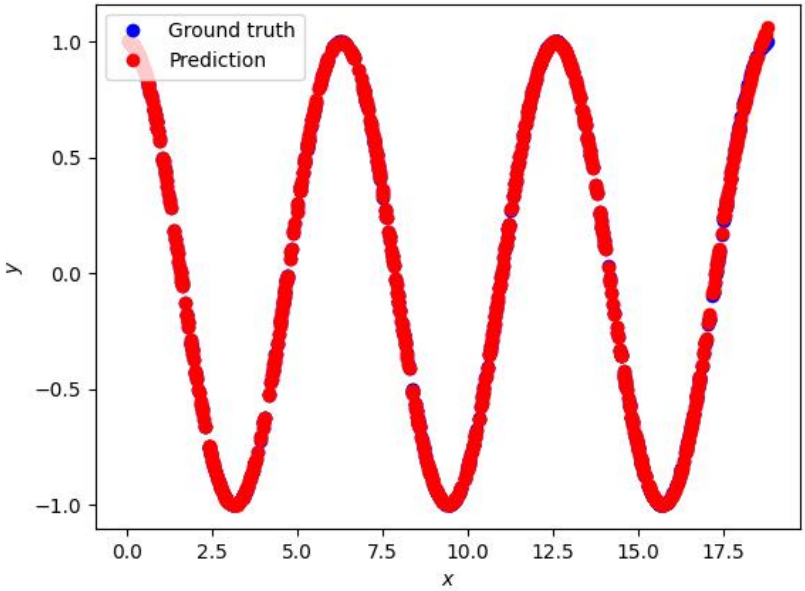

Рисунок 15 – Аппроксимация после 50000 (финал) эпох (n=1000)

При n = 1000 сеть достигает практически идеальной аппроксимации. Уже после 10000 эпох предсказания сети почти не отличаются от эталонной функции. Это подтверждает, что для данной задачи 1000 обучающих примеров является достаточным количеством.

5. Выводы по дополнительному заданию

Размер обучающей выборки критически влияет на качество аппроксимации нейронной сети. При недостаточном количестве данных (n = 10) сеть не может обучиться даже после большого числа эпох.

Увеличение выборки с 10 до 100 примеров даёт существенное улучшение — RMSE снижается примерно в 2-3 раза в зависимости от конфигурации сети.

Наибольшее улучшение наблюдается при увеличении выборки с 100 до 1000 примеров — для сети с 10 нейронами RMSE снижается с 0.08 до 0.012 (в 6.7 раза).

Для сложных нелинейных функций, таких как синусоида, требуется достаточно большой объем данных (не менее 1000 примеров) для достижения высокого качества аппроксимации.

Динамика обучения показывает, что даже при достаточном количестве данных (n = 1000) сети требуется значительное число эпох (10000-20000) для достижения приемлемого качества.

6. Выводы по работе

В ходе выполнения лабораторной работы была исследована способность многослойных нейронных сетей к аппроксимации нелинейной функции $y = \sin(x + \pi/2)$.

Основные выводы:

1. Нейронные сети успешно аппроксимируют нелинейные функции. При достаточном количестве нейронов (10 и более) сеть способна практически точно воспроизвести сложную синусоидальную зависимость.
2. Количество нейронов важнее количества слоёв. Увеличение числа нейронов с 2 до 10 снижает ошибку RMSE в десятки раз, тогда как добавление второго скрытого слоя даёт улучшение лишь в 3-5 раз.
3. Размер обучающей выборки критически влияет на качество обучения. При n=10 сеть не может обучиться даже после 50000 эпох. Увеличение выборки до 1000 примеров снижает RMSE с 0,28 до 0,012 — улучшение в 23 раза.
4. Оптимальная конфигурация по точности: 2 скрытых слоя по 10 нейронов (RMSE = 0,0036). Оптимальная по сложности/точности: 1 скрытый слой из 10 нейронов (RMSE = 0,0121).
5. Для качественного обучения нелинейных функций требуется не менее 1000 примеров — только при таком объёме данных сеть достигает практически идеальной аппроксимации.

ПРИЛОЖЕНИЕ А

В данном приложении представлен модифицированный код программы на языке Python, реализующий создание, обучение и тестирование многослойной нейронной сети для аппроксимации функции y = sin(x + π/2). Код включает функции генерации данных, построения сети, обучения с выводом промежуточных результатов и визуализации графиков аппроксимации.

```
import math
import torch
import numpy as np
import matplotlib.pyplot as plt

# ----------------------------
# 1. Генерация данных
# ----------------------------
def generate_data(n_samples, x_range=(0, 6*math.pi)):
    x = torch.rand(n_samples) * x_range[1]
    y = torch.sin(x + math.pi/2)
    return x, y

# Проверочная выборка (отсортирована для красивого графика)
x_test = torch.linspace(0, 6*math.pi, 500)
y_test = torch.sin(x_test + math.pi/2)

# ----------------------------
# 2. Создание сети с заданным числом скрытых слоёв
# ----------------------------
class Network(torch.nn.Module):
    def __init__(self, hidden_neurons, num_hidden_layers=2):
        super(Network, self).__init__()
        self.num_hidden_layers = num_hidden_layers
        layers = []
        # Входной слой
        layers.append(torch.nn.Linear(1, hidden_neurons))
        layers.append(torch.nn.Sigmoid())
        # Промежуточные скрытые слои
        for _ in range(num_hidden_layers - 1):
            layers.append(torch.nn.Linear(hidden_neurons, hidden_neurons))
            layers.append(torch.nn.Sigmoid())
        # Выходной слой
        layers.append(torch.nn.Linear(hidden_neurons, 1))
        self.net = torch.nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x)

# ----------------------------
# 3. Функция потерь и визуализации
# ----------------------------
def loss(pred, target):
    return ((pred - target) ** 2).mean()

def predict_and_plot(net, x_test, y_test, epoch, hidden_neurons, num_layers,
save=True):
    net.eval()
    with torch.no_grad():
        y_pred = net(x_test.unsqueeze(1)).squeeze()
```

```
    plt.figure(figsize=(8, 5))
    plt.plot(x_test.numpy(), y_test.numpy(), 'b-', label='Ground truth')
    plt.plot(x_test.numpy(), y_pred.numpy(), 'r--', label='Prediction')
    plt.legend()
    plt.title(f'Layers={num_layers}, Neurons={hidden_neurons}, Epoch={epoch}')
    plt.xlabel('x')
    plt.ylabel('y')
    if save:
        plt.savefig(f'graph_L{num_layers}_N{hidden_neurons}_epoch{epoch}.png')
    else:
        plt.show()
    plt.close()
    return y_pred

# ----------------------------
# 4. Обучение сети
# ----------------------------
def train_network(hidden_neurons, num_hidden_layers, n_train, epochs=50001, lr=0.01):
    # Генерация обучающих данных
    x_train, y_train = generate_data(n_train)
    x_train = x_train.unsqueeze(1)
    y_train = y_train.unsqueeze(1)

    net = Network(hidden_neurons, num_hidden_layers)
    optimizer = torch.optim.Adam(net.parameters(), lr=lr)

    for i in range(epochs):
        optimizer.zero_grad()
        y_pred = net(x_train)
        loss_val = loss(y_pred, y_train)
        loss_val.backward()
        optimizer.step()

        if i % 5000 == 0:
            with torch.no_grad():
                y_pred_test = net(x_test.unsqueeze(1))
                rmse = torch.sqrt(((y_pred_test.squeeze() - y_test) ** 2).mean())
                print(f'Layers={num_hidden_layers}, Neurons={hidden_neurons},
Epoch={i:5d}, Loss={loss_val.item():.6f}, RMSE={rmse.item():.6f}')
            predict_and_plot(net, x_test, y_test, i, hidden_neurons,
num_hidden_layers)

    # Финальная RMSE
    with torch.no_grad():
        y_pred_final = net(x_test.unsqueeze(1)).squeeze()
        final_rmse = torch.sqrt(((y_pred_final - y_test) ** 2).mean())
        print(f'\n=== Финальная RMSE: {final_rmse.item():.6f} ===')
    return net, final_rmse.item()

# ----------------------------
# 5. Эксперименты
# ----------------------------
if __name__ == "__main__":
    # Параметры эксперимента
    hidden_neurons_list = [2, 3, 5, 10]   # число нейронов в каждом скрытом слое
    num_layers_list = [1, 2]              # 1 скрытый слой и 2 скрытых слоя
    n_train = 1000                        # размер обучающей выборки

    results = {}
    for num_layers in num_layers_list:
        print(f"\n{'='*60}")
```

```
    print(f"Эксперимент: {num_layers} скрытый(ых) слой(ев)")
    for hn in hidden_neurons_list:
        print(f"\n--- Нейронов в каждом скрытом слое: {hn} ---")
        net, rmse = train_network(hn, num_layers, n_train, epochs=50001, lr=0.01)
        results[(num_layers, hn)] = rmse

# Вывод сводной таблицы RMSE
print("\n\nСВОДНАЯ ТАБЛИЦА ФИНАЛЬНОЙ RMSE (на тестовой выборке):")
print("Слои \ Нейроны\t" + "\t".join(map(str, hidden_neurons_list)))
for num_layers in num_layers_list:
    row = f"{num_layers}\t\t"
    for hn in hidden_neurons_list:
        row += f"{results.get((num_layers, hn), 0):.6f}\t"
    print(row)
```